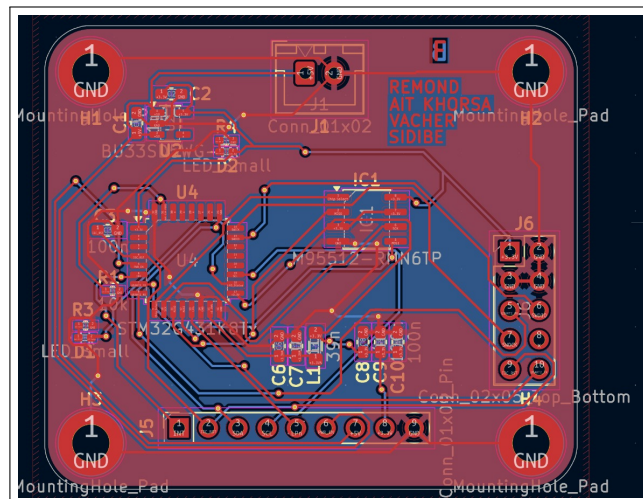


Conception, réalisation et validation d'un système embarqué
de détection et de comptage bidirectionnel de personnes



Période : Février 2026 — Mai 2026
Volume horaire : 36 heures (9 séances × 4h) + 2 séances TD KiCad
Date de rendu : 18 mai 2026

Abstract

This project presents the design and implementation of an autonomous presence counting system based on the **VL53L5CX** Time-of-Flight (ToF) matrix sensor, built around an **STM32G431KBT6** microcontroller on a custom two-layer PCB designed with KiCad.

The sensor provides an 8×8 depth map at up to 15 Hz, enabling the detection of individuals moving through a doorway. A **finite-state machine (FSM)** implemented in C analyses column-averaged distance data to distinguish left-to-right entries from right-to-left exits, maintaining a running occupancy counter. The counter is saved to a non-volatile **M95512-RMN6TP** SPI EEPROM so the count persists across power cycles.

Hardware integration includes a 3.3 V LDO power supply (**BU33SD5WG-TR**), decoupling capacitors, GPIO biasing resistors, LED current-limiting resistors, a SWD programming interface, and diagnostic LEDs. JLC PCB manufacturing constraints (0.15 mm minimum trace width, 0.45 mm via diameter) were respected throughout. Functional validation was performed block by block : power supply, MCU boot, LED toggling, and sensor communication. One routing error (wrong I²C pin assignment) was identified, documented, and corrected.

Keywords : people counting, VL53L5CX, Time-of-Flight, STM32G431, KiCad, PCB, I²C, SPI, EEPROM, finite-state machine, embedded systems.

Table des matières

Abstract	1
1 Introduction	3
2 Cahier des charges et architecture globale	4
2.1 Objectif fonctionnel	4
2.2 Spécifications techniques	4
2.3 Architecture fonctionnelle	4
3 Étude des composants principaux	5
3.1 Microcontrôleur : STM32G431KBT6	5
3.1.1 Présentation et justification	5
3.1.2 Découplage et alimentation	5
3.1.3 Mapping des broches	5
3.2 Capteur ToF : VL53L5CX	5
3.2.1 Principe de fonctionnement	5
3.2.2 Interfaçage matériel	6
3.3 Mémoire EEPROM : M95512-RMN6TP	6
3.4 Régulateur de tension : BU33SD5WG-TR	6
4 Conception du schéma électronique sous KiCad	7
4.1 Démarche de conception	7
4.1.1 Sélection et import des composants	7
4.1.2 Organisation par blocs fonctionnels	7
4.1.3 Gestion de la connectivité par labels	7
4.1.4 Vérification ERC et power flags	7
4.2 Schéma final	7
4.2.1 Points de conception notables	7
5 Routage PCB et réalisation matérielle	9
5.1 Contraintes de fabrication	9
5.2 Stratégie de placement et de routage	9
5.3 Description du PCB routé	9
5.4 Erreur de routage identifiée : mauvais pin I ² C	10
5.5 Soudage et assemblage	10
5.5.1 Préparation du pochoir et dépôt de pâte	10
5.5.2 Pose des composants CMS	10
5.5.3 Passage au four de refusion	11
5.5.4 Soudage manuel des connecteurs	11
5.5.5 Contrôles après soudage	11
6 Interfaces matériel/logiciel	12
6.1 Interface de programmation SWD	12
6.2 Interface capteur ToF — I ² C	12
6.3 Interface EEPROM — SPI	12

7	Analyse du fonctionnement logiciel	13
7.1	Organisation générale	13
7.2	Fonctions principales utilisées	13
7.3	Initialisation du capteur	13
7.4	Analyse des colonnes de la matrice	13
7.5	Machine à états finis	14
7.6	Limites identifiées	14
8	Tests et validation	15
8.1	Méthodologie	15
8.2	Test 1 — Alimentation 3,3V	15
8.3	Test 2 — Démarrage du STM32 et clignotement LED	15
8.4	Test 3 — Communication I ² C avec le capteur VL53L5CX	15
8.5	Test 4 — Algorithme de comptage FSM	15
8.6	Récapitulatif	16
9	Bilan, conseils et conclusion	17
9.1	Chronologie du projet	17
9.2	Trois conseils positifs pour nos successeurs	17
9.3	Trois points à éviter	17
9.4	Conclusion	17
	Références principales	18

1. Introduction

Dans un contexte de transition écologique, les bâtiments intelligents représentent un levier majeur pour réduire la consommation énergétique. Connaître en temps réel le nombre d'occupants d'un espace permet d'automatiser l'éclairage, le chauffage ou la ventilation, et d'éviter de gaspiller de l'énergie dans des salles vides.

Notre groupe a été chargé, dans le cadre du mini-projet d'électronique 1AB de l'ENSEA, de concevoir un **système autonome de comptage de présence**. L'enjeu était de parcourir l'intégralité du cycle de développement d'un système embarqué : du cahier des charges au prototype physique validé, en neuf séances de quatre heures.

Le choix technologique s'est porté sur le capteur **VL53L5CX** de STMicroelectronics, un capteur Time-of-Flight matriciel 8×8 . Contrairement à une caméra, il ne capture aucune image reconnaissable, ce qui simplifie les problématiques de confidentialité. Installé sur un mur, à proximité d'un point de passage, il observe une zone sous forme de matrice de distances 8×8 . L'évolution de cette matrice permet de détecter le passage d'une personne et d'en déduire le sens de déplacement.

Ce rapport décrit la démarche technique complète suivie par le groupe :

- **Chapitre 2** — Cahier des charges et architecture globale
- **Chapitre 3** — Étude des composants principaux
- **Chapitre 4** — Conception du schéma électronique sous KiCad
- **Chapitre 5** — Routage PCB et réalisation matérielle
- **Chapitre 6** — Interfaces matériel/logiciel
- **Chapitre 7** — Analyse du fonctionnement logiciel
- **Chapitre 8** — Tests et validation
- **Chapitre 9** — Bilan, conseils et conclusion

2. Cahier des charges et architecture globale

2.1 Objectif fonctionnel

Le système doit **comptabiliser en temps réel le nombre de personnes présentes** dans une zone délimitée. Placé au-dessus d'un point de passage (porte, couloir), il détecte les individus qui traversent et maintient un compteur d'occupation :

- **+1** lorsqu'une personne entre (passage de gauche à droite vu du capteur) ;
- **-1** lorsqu'une personne sort (passage de droite à gauche).

Le compteur doit être persistant : en cas de coupure d'alimentation, la valeur est sauvegardée en mémoire non-volatile et restaurée au redémarrage.

2.2 Spécifications techniques

Paramètre	Valeur / Choix retenu
Alimentation entrée	5 V (connecteur J1)
Tension interne	3,3 V (régulateur LDO BU33SD5WG-TR)
Microcontrôleur	STM32G431KBT6 (Cortex-M4F, 170 MHz, 128 kB Flash, 32 kB SRAM)
Capteur ToF	VL53L5CX, matrice 8×8, jusqu'à 15 Hz
Seuil de détection	1 000 mm (paramètre DIST_THRESHOLD_MM)
Interface capteur	I ² C2, adresse 0x52
Mémoire non-volatile	EEPROM M95512-RMN6TP, 512 Kbit, SPI
Interface programmation	SWD (connecteur J6, compatible sonde Nucleo ENSEA)
PCB	2 couches, fabrication JLC PCB

TABLE 2.1 – Spécifications techniques du système

2.3 Architecture fonctionnelle

L'architecture s'organise autour de cinq blocs fonctionnels. Le STM32 est au centre et orchestre tous les échanges :

1. **Alimentation** — Régulation 5 V → 3,3 V (BU33SD5WG-TR + condensateurs de filtrage)
2. **Cerveau** — STM32G431KBT6 (traitement FSM, coordination des périphériques)
3. **Détection** — VL53L5CX via connecteur J5 (matrice 8×8, I²C)
4. **Stockage** — EEPROM M95512-RMN6TP (persistance du compteur, SPI)
5. **Débogage/IHM** — SWD (J6), LEDs D1/D2 avec résistances de protection

3. Étude des composants principaux

3.1 Microcontrôleur : STM32G431KBT6

3.1.1 Présentation et justification

Le **STM32G431KBT6** est le cœur du système. Il embarque un processeur ARM Cortex-M4 à 170 MHz avec unité de calcul flottant (FPU), 128 kB de Flash et 32 kB de SRAM. Son boîtier **LQFP32** compact offre suffisamment de broches pour gérer simultanément : l'I²C vers le capteur, le SPI vers l'EEPROM, les LEDs et l'interface SWD de programmation, tout en limitant l'encombrement du PCB.

3.1.2 Découplage et alimentation

Conformément à la fiche technique ST, des condensateurs de **100 nF** (C3, C9, C10) ont été placés au plus près de chaque paire VDD/VSS. Ces condensateurs filtrent les perturbations haute fréquence et évitent les resets intempestifs lors des commutations rapides des périphériques.

3.1.3 Mapping des broches

Broche(s)	Périphérique	Fonction
PA8 / PA9	I2C2 SDA / SCL	Communication avec VL53L5CX
PB3 / PB4 / PB5	SPI1 SCK/MISO/MOSI	Communication avec EEPROM M95512
PA4	GPIO Output	Chip Select (CS) EEPROM
PA13 / PA14	SWD SWDIO / SWCLK	Programmation et débogage
PB0	GPIO Output	Reset / LPn du capteur (réveil)
PB1	EXTI1 IT_FALLING	Interruption INT du capteur
PA5 / PA6	GPIO Output	LEDs D1 et D2 (statut)

TABLE 3.1 – Mapping des broches du STM32G431KBT6

3.2 Capteur ToF : VL53L5CX

3.2.1 Principe de fonctionnement

Le **VL53L5CX** est un capteur laser Time-of-Flight multizone de STMicroelectronics. Il projette un faisceau laser infrarouge structuré et mesure le temps de retour pour chacun des 64 points de sa grille, reconstituant une **carte de profondeur 8×8** avec une précision millimétrique.

À 15 Hz, il fournit 15 images de profondeur par seconde, ce qui est largement suffisant pour suivre un individu marchant à vitesse normale. Contrairement à un capteur PIR (infrarouge passif), il ne détecte pas seulement une présence mais fournit des **mesures de distance exploitables** pour analyser la trajectoire et déterminer le sens de passage.

3.2.2 Interfaçage matériel

La communication avec le STM32 s'effectue via le bus **I²C à l'adresse 0x52**. Deux signaux de contrôle complètent l'interface :

- **LPn** (PB0, sortie) : permet de réinitialiser le capteur au démarrage ;
- **INT** (PB1, interruption EXTI1) : signale la disponibilité de nouvelles données.

Dans cette révision du schéma, **R1 (10 kΩ)** n'est pas une résistance de tirage du bus I²C : elle est reliée entre la broche PB8 du STM32 et la masse afin d'imposer un niveau bas par défaut et d'éviter une entrée flottante. Les lignes SDA et SCL du bus I²C ne sont donc pas polarisées par R1. Leur tirage au 3,3 V doit être assuré par les résistances présentes sur le module VL53L5CX-SATEL ou ajouté dans une révision du PCB si le module utilisé n'en possède pas.

3.3 Mémoire EEPROM : M95512-RMN6TP

Le **M95512-RMN6TP** est une EEPROM série 512 Kbit (64 ko) de STMicroelectronics. Elle communique avec le STM32 via **SPI** (SCK, MISO, MOSI, CS). Son rôle est de **sauvegarder le compteur de présence** : même après une coupure d'alimentation, la valeur est restaurée au redémarrage.

L'interface SPI a été préférée à l'I²C pour sa rapidité en écriture, ce qui minimise la latence lors de chaque mise à jour du compteur.

3.4 Régulateur de tension : BU33SD5WG-TR

Le **BU33SD5WG-TR** est un régulateur LDO qui abaisse proprement la tension d'entrée de 5 V à **3,3 V** avec un courant de sortie maximal de 500 mA. Le montage suit les recommandations de la fiche technique :

- C1 = 1 μF en entrée pour stabiliser la source ;
- C2 = 1 μF et C8 = 4 μF en sortie pour filtrer les appels de courant ;
- Broche STBY reliée au 5 V pour maintenir le régulateur actif en permanence.

4. Conception du schéma électronique sous KiCad

4.1 Démarche de conception

La conception du schéma a suivi une approche **modulaire et progressive**, permettant de travailler en parallèle sur les blocs fonctionnels et de vérifier chacun indépendamment.

4.1.1 Sélection et import des composants

Les symboles des composants principaux ont été importés depuis les bibliothèques KiCad standards (STM32G431KBTx, résistances, condensateurs, connecteurs). Pour l'EEPROM M95512, le symbole et l'empreinte ont été téléchargés depuis la plateforme **Ultra Librarian via Farnell** au format KiCad, puis intégrés à la bibliothèque locale du projet.

4.1.2 Organisation par blocs fonctionnels

Pour garantir un schéma lisible et professionnel, nous avons utilisé des **cadres en pointillés** pour regrouper les composants par fonction : Régulateur de tension, Microcontrôleur STM32, EEPROM, Connecteur capteur VL53L5CX, Connecteur SWD et Trous de fixation.

4.1.3 Gestion de la connectivité par labels

Plutôt que de tracer des fils traversant tout le schéma, nous avons utilisé des **labels nets** nommés (SDA, SCL, MISO, MOSI, SCK, INT, LPn). KiCad relie automatiquement toutes les broches portant le même nom, ce qui simplifie la lecture du schéma et facilite la vérification ERC.

4.1.4 Vérification ERC et power flags

Avant de passer au routage, l'outil de **vérification des règles électriques (ERC)** a été lancé. Les symboles PWR_FLAG ont été ajoutés sur les nœuds +5V, +3.3V et GND pour éviter les fausses alertes ERC liées aux sources de puissance non reconnues.

4.2 Schéma final

Le schéma final est organisé autour des blocs fonctionnels décrits précédemment. L'image complète du schéma KiCad n'est pas reproduite dans le corps du rapport afin d'alléger le document et de conserver uniquement l'illustration générale du PCB en page de garde. Le fichier KiCad fourni séparément permet de consulter le schéma complet, les empreintes et les connexions exactes si une modification doit être faite par la suite.

La lecture du schéma se fait de gauche à droite : l'alimentation 5V arrive sur le connecteur d'entrée, le régulateur fournit le 3,3V, puis cette tension alimente le STM32, l'EEPROM et les lignes de communication. Les connecteurs externes sont volontairement séparés du microcontrôleur pour faciliter les tests au multimètre et à l'oscilloscope.

4.2.1 Points de conception notables

- **Condensateurs de découplage** : Chaque VDD du STM32 est découplé par un condensateur 100 nF (C3, C9, C10) pour filtrer les perturbations HF.

-
- **Polarisation de PB8** : R1 ($10\text{ k}\Omega$) est reliée entre PB8 et GND. Elle fixe un niveau bas par défaut sur cette broche et évite un état flottant. Elle ne doit donc pas être interprétée comme une résistance de pull-up du bus I²C.
 - **Protection des LEDs** : D1 et D2 sont protégées par R3 et R4 ($1\text{ k}\Omega$), calculées pour un courant d'environ 2 mA à 3,3 V.
 - **Connecteur SWD J6** : Connecteur 2×5 broches (SWDIO, SWCLK, RESET, I2C_RST, USART2_TX/RX) compatible sonde Nucleo ENSEA.
 - **Connecteur capteur J5** : Connecteur 9 broches vers le module VL53L5CX-SATEL externe (INT, I2C_RST, SDA, SCL, LPn, NB_EN, +5V, +3.3V, GND).

5. Routage PCB et réalisation matérielle

5.1 Contraintes de fabrication

Le routage respecte les contraintes JLC PCB 2026 pour 2 couches :

Paramètre	Valeur minimale
Largeur de piste	0,15 mm
Isolation (clearance)	0,15 mm
Largeur minimale de connexion	0,15 mm
Largeur minimale d'anneau	0,125 mm
Diamètre minimum de via	0,45 mm
Isolation trou/cuivre	0,30 mm
Isolation trou à trou	0,50 mm

TABLE 5.1 – Contraintes de fabrication JLC PCB (importées dans KiCad)

5.2 Stratégie de placement et de routage

Placement central du STM32 : Le microcontrôleur est positionné au centre de la carte pour minimiser la longueur des pistes vers tous les périphériques.

Découplage au plus près : Les condensateurs de découplage sont routés directement au plus près des broches VDD correspondantes, conformément à la recommandation ST.

Gestion des couches :

- **Face Top** (couche rouge) : composants CMS et pistes principales ;
- **Face Bottom** (couche bleue) : croisements de pistes et plan de masse GND ;
- **Vias** : utilisés pour changer de couche et connecter au plan de masse.

Plan de masse : Un plan de masse GND est appliqué sur les deux couches pour réduire les émissions électromagnétiques et améliorer la stabilité des tensions.

Mécanique : Quatre trous de fixation M3 (H1 à H4) aux coins de la carte. La sérigraphie inclut les noms du groupe.

5.3 Description du PCB routé

La vue du PCB est conservée en page de garde pour éviter de multiplier les captures KiCad dans le rapport. Le routage final suit une logique simple : les connecteurs sont placés sur les bords de la carte pour faciliter le branchement, tandis que le STM32 reste au centre afin de limiter la longueur des pistes critiques. Les alimentations et le plan de masse sont répartis sur les deux couches, ce qui améliore la stabilité électrique et simplifie le retour de courant.

Les pistes de communication I²C vers le capteur VL53L5CX ont été routées avec une attention particulière, car elles servent à récupérer les matrices de distance. Le bus SPI de l'EEPROM a également été gardé court pour limiter les perturbations. Les vias sont utilisés uniquement lorsque le changement de couche est nécessaire pour éviter un croisement ou rejoindre le plan de masse.

Après le routage, une vérification DRC a été réalisée avec les contraintes JLC PCB. Les principaux points contrôlés ont été la largeur minimale des pistes, l'espacement entre deux pistes, le diamètre des vias et la distance entre le cuivre et les trous de fixation. Cette étape est indispensable avant l'export des fichiers Gerber, car elle permet d'éviter une carte impossible à fabriquer ou présentant des courts-circuits.

5.4 Erreur de routage identifiée : mauvais pin I²C

Problème — Erreur de mapping I²C (broche PA15 au lieu de PA8)

Lors de la configuration du projet dans STM32CubeIDE, nous avons découvert une erreur de conception dans le schéma KiCad : le signal I2C2_SDA avait été associé à la broche **PA15**, alors que sur le STM32G431KBT6, la ligne SDA du périphérique I2C2 est physiquement fixée sur **PA8**.

Cette erreur s'est traduite par une absence totale de communication avec le capteur lors des premiers tests (pas de réponse à l'adresse 0x52).

Solution adoptée : Un fil volant a été soudé entre PA8 et le pad SDA du connecteur J5. Cette correction sera intégrée dans la révision 1.1 du schéma KiCad.

5.5 Soudage et assemblage

L'assemblage a été réalisé en plusieurs étapes afin de limiter les erreurs et de pouvoir contrôler la carte avant la première mise sous tension. Avant de souder, nous avons d'abord vérifié le PCB reçu : état général des pistes, propreté des pads, absence de rayures importantes et cohérence de l'orientation des composants. Cette vérification est importante, car une erreur de placement ou un défaut visible peut être beaucoup plus difficile à corriger une fois les composants soudés.

5.5.1 Préparation du pochoir et dépôt de pâte

Pour les composants CMS, nous avons utilisé des rails imprimés afin de maintenir correctement le pochoir au-dessus du PCB. Ces rails servent de guide mécanique : ils empêchent le pochoir de bouger pendant l'application de la pâte à braser. Les zones grises du pochoir correspondent aux ouvertures par lesquelles la pâte est déposée sur les pads du circuit imprimé.

La pâte à braser a été étalée en couche fine et régulière. Il faut éviter d'en mettre trop, car un excès de pâte peut créer des ponts de soudure entre les broches du STM32. À l'inverse, une quantité insuffisante peut provoquer une soudure incomplète. Après retrait du pochoir, nous avons contrôlé visuellement que la pâte était bien déposée sur les empreintes, en particulier autour du STM32 et de l'EEPROM.

5.5.2 Pose des composants CMS

Les composants ont ensuite été placés manuellement à l'aide d'une pince fine. Nous avons commencé par les petits composants passifs, comme les résistances et les condensateurs, puis nous avons placé les circuits intégrés : régulateur, EEPROM et STM32. Le repère de la broche 1 du STM32 a été vérifié avant la pose, car une mauvaise orientation du microcontrôleur rendrait la carte inutilisable.

Cette étape demande de la précision, mais elle permet aussi de vérifier que les empreintes choisies dans KiCad correspondent bien aux composants réels. Le STM32 en boîtier LQFP32 est le composant le plus délicat, car ses broches sont rapprochées. Un mauvais alignement peut provoquer un contact entre deux broches ou une broche non soudée.

5.5.3 Passage au four de refusion

Une fois les composants CMS positionnés, la carte a été envoyée au four de refusion. Pendant cette étape, la pâte à braser fond, se répartit sur les pads, puis se solidifie en refroidissant. Cette méthode permet de souder en une seule fois les composants CMS avec un résultat plus régulier qu'une soudure manuelle broche par broche.

Après le passage au four, nous avons inspecté les soudures à la loupe. Les contrôles principaux ont porté sur l'absence de ponts entre broches, le bon alignement des composants et la qualité visuelle des soudures. Une attention particulière a été portée au STM32, à l'EEPROM et aux condensateurs de découplage proches des alimentations.

5.5.4 Soudage manuel des connecteurs

Les connecteurs traversants, notamment J1, J5 et J6, n'ont pas été soudés au four. Ils ont été soudés manuellement au fer, car ils sont plus volumineux et doivent rester bien alignés mécaniquement. Pour éviter un connecteur incliné, nous avons d'abord soudé une seule broche, vérifié l'alignement, puis soudé les autres broches.

Les éventuelles reprises de routage, comme le fil volant utilisé pour corriger la ligne SDA de l'I²C, ont également été faites au fer à souder. Cette correction permet de poursuivre les tests sans recommander immédiatement une nouvelle carte. Elle reste cependant une solution provisoire : la version suivante du PCB devra intégrer directement la correction dans le routage KiCad.

5.5.5 Contrôles après soudage

Avant la première mise sous tension, un test de continuité au multimètre a été réalisé entre les rails 5 V, 3,3 V et GND. L'objectif était de vérifier qu'il n'y avait pas de court-circuit entre les alimentations. Nous avons aussi contrôlé quelques connexions importantes : alimentation du STM32, masse des connecteurs, et lignes de communication principales.

La première mise sous tension a été faite progressivement. La sortie du régulateur a été mesurée avant de brancher le programmeur, puis le STM32 a été flashé avec un programme simple de clignotement de LED. Cette méthode réduit le risque d'endommager la carte, car chaque bloc est validé avant de passer au suivant.

Au final, l'assemblage a combiné deux méthodes complémentaires : la refusion au four pour les composants CMS et la soudure manuelle pour les connecteurs et les reprises. Cette organisation est adaptée à notre carte, car elle permet d'obtenir des soudures propres sur les petits composants tout en gardant un bon contrôle mécanique sur les connecteurs.

6. Interfaces matériel/logiciel

6.1 Interface de programmation SWD

Le connecteur J6 (2×5 broches) relie les broches SWDIO (PA13) et SWCLK (PA14) du STM32 à la sonde de débogage d’une carte Nucleo ENSEA. Ce pont matériel permet de **flasher le firmware** depuis STM32CubeIDE via *Run* → *Debug* ou *Run* → *Flash*. La broche RESET est également routée pour permettre une réinitialisation matérielle à distance sans avoir à appuyer sur un bouton physique.

6.2 Interface capteur ToF — I²C

La communication avec le VL53L5CX utilise le périphérique **I2C2** du STM32, configuré avec les paramètres suivants (registre Timing : 0x30D29DE4) :

- Mode d’adressage 7 bits, filtre analogique activé, filtre numérique à 0 ;
- Adresse esclave VL53L5CX : 0x52 ;
- La broche LPn (PB0) pilote le réveil/reset physique du capteur ;
- La broche INT (PB1) est configurée en EXTI1 (front descendant) pour une évolution future vers un mode basse consommation par interruption.

6.3 Interface EEPROM — SPI

L’EEPROM est connectée au bus **SPI1**. Chaque mise à jour du compteur déclenche une séquence SPI : activation du CS (PA4 à l’état bas), envoi de la commande *Write Enable*, écriture de la donnée, attente de fin d’écriture (*Read Status Register*), désactivation du CS. Le Chip Select est géré manuellement en GPIO pour un contrôle précis des transactions.

7. Analyse du fonctionnement logiciel

7.1 Organisation générale

Le code complet n'est pas reproduit dans ce rapport afin d'éviter de surcharger le document : il est fourni séparément dans le dépôt Git ou dans l'archive ZIP du projet. Ici, on présente uniquement l'analyse du fonctionnement, les fonctions utilisées et le rôle de chacune dans la chaîne de comptage.

Le firmware est développé sous **STM32CubeIDE**. Les fonctions générées par CubeMX configurent d'abord les périphériques du STM32, puis le programme applicatif initialise le capteur VL53L5CX et lance une boucle principale de mesure. Le principe général est le suivant : acquisition d'une matrice de distances 8×8 , extraction de quelques colonnes utiles, décision par seuil, puis mise à jour du compteur par machine à états finis.

7.2 Fonctions principales utilisées

Fonction / variable	Rôle dans le projet
<code>main()</code>	Lance les initialisations, vérifie la présence du capteur et exécute la boucle de comptage.
<code>SystemClock_Config()</code>	Configure l'horloge du STM32 pour obtenir une fréquence de fonctionnement stable.
<code>MX_GPIO_Init()</code>	Configure les broches de contrôle : LPn, INT, LEDs et Chip Select.
<code>MX_I2C2_Init()</code>	Configure le bus I ² C2 utilisé pour communiquer avec le VL53L5CX.
<code>Get_Column_Average()</code>	Calcule la distance moyenne d'une colonne de la matrice 8×8 en gardant uniquement les mesures fiables.
<code>currentState</code>	Mémoire l'état actuel de la machine à états finis.
<code>passageCounter</code>	Contient la valeur du compteur de présence.
<code>Error_Handler()</code>	Bloque le programme en cas d'erreur critique détectée par la HAL.

TABLE 7.1 – Fonctions et variables importantes du firmware

7.3 Initialisation du capteur

Au démarrage, le STM32 active d'abord ses périphériques internes, puis il réveille le capteur par la broche LPn. Ensuite, le programme vérifie que le VL53L5CX répond bien sur le bus I²C à l'adresse 0x52. Si le capteur est détecté, il est configuré en résolution 8×8 avec une fréquence de mesure de 15 Hz.

Cette fréquence est suffisante pour notre application, car un passage humain devant le capteur dure plusieurs centaines de millisecondes. On obtient donc plusieurs mesures pendant un même déplacement, ce qui permet de suivre l'évolution de la position dans la matrice.

7.4 Analyse des colonnes de la matrice

Le VL53L5CX renvoie 64 distances, organisées comme une image de profondeur. Pour éviter de traiter toute la matrice à chaque itération, le code ne s'appuie que sur quelques colonnes

représentatives du déplacement. La fonction `Get_Column_Average()` calcule la moyenne d'une colonne, mais seulement si les mesures sont considérées comme fiables par l'API du capteur.

Colonne	Interprétation	Utilisation
0	Bord gauche de la zone observée	Validation d'une sortie vers la gauche
3	Zone gauche proche du centre	Début d'un passage gauche → droite
4	Zone droite proche du centre	Début d'un passage droite → gauche
7	Bord droit de la zone observée	Validation d'une sortie vers la droite

TABLE 7.2 – Colonnes utilisées par l'algorithme de détection

Une présence est considérée comme détectée lorsqu'une colonne donne une distance inférieure au seuil fixé, par exemple `DIST_THRESHOLD_MM = 1000`. Ce choix permet d'écarter les éléments trop éloignés du champ de passage, comme un mur ou un objet placé en arrière-plan.

7.5 Machine à états finis

La décision de comptage repose sur une machine à états finis à trois états. L'état `STATE_IDLE` correspond à l'absence de passage en cours. Si la colonne 3 est franchie en premier, le programme attend ensuite une confirmation sur la colonne 7 : le passage est alors interprété comme une entrée et le compteur est incrémenté. À l'inverse, si la colonne 4 est franchie en premier puis la colonne 0, le passage est interprété comme une sortie et le compteur est décrémenté.

Intérêt de la FSM : le compteur n'est pas modifié dès qu'un objet apparaît. Il faut observer une succession cohérente de colonnes. Cela évite de compter une personne immobile, une main qui s'arrête devant le capteur ou un objet présent dans le champ sans traverser réellement la zone.

7.6 Limites identifiées

L'algorithme fonctionne correctement dans le cas prévu par le cahier des charges : un passage à la fois, dans un sens clairement identifiable. En revanche, il reste limité si deux personnes passent simultanément ou si une personne change de sens au milieu du champ. Pour une version plus avancée, il faudrait suivre plusieurs zones en parallèle, ajouter un filtrage temporel plus complet et éventuellement mémoriser une trajectoire au lieu de seulement détecter un ordre de colonnes.

8. Tests et validation

8.1 Méthodologie

Conformément aux recommandations du cahier des charges, nous avons adopté une approche **bloc par bloc** : valider chaque partie du système indépendamment avant de les intégrer. Cette stratégie nous a permis d'identifier rapidement l'erreur de routage I²C sans avoir à déboguer l'ensemble du système simultanément.

8.2 Test 1 — Alimentation 3,3 V

Procédure : Alimentation de la carte en 5 V via J1, mesure au multimètre des nœuds +5V et +3.3V (Power Flags du schéma).

Résultat : Tension mesurée en sortie du régulateur BU33SD5WG-TR : **3,30 V ± 0,02 V**. Tension d'entrée stable à 5,00 V. Aucun composant n'a chauffé anormalement. **Test validé.**

8.3 Test 2 — Démarrage du STM32 et clignotement LED

Procédure : Configuration de la broche PA5 (LED D1) en GPIO Output dans STM32CubeIDE, flash d'un programme de clignotement simple (HAL_GPIO_TogglePin toutes les 500 ms).

Résultat : Flash réussi sans erreur. LED D1 clignote à la fréquence attendue (1 Hz). Le STM32 est soudé correctement et opérationnel. **Test validé.**

8.4 Test 3 — Communication I²C avec le capteur VL53L5CX

Procédure : Appel à HAL_I2C_IsDeviceReady(&hi2c2, 0x52, 5, 100) et affichage du résultat sur la console série USART2.

Problème : Pas de réponse à l'adresse 0x52. L'oscilloscope n'a montré aucun signal sur PA15 (broche erronément câblée). Après identification de l'erreur de routage et pose d'un fil volant sur PA8, nous nous sommes rendus compte d'une erreur sur le MCU : le composant soudé sur le deuxième PCB était un STM32G441KBT6 au lieu du STM32G431KBT6 prévu. Le programme ne peut donc toujours pas détecter correctement le capteur.

Statut : **Corrigé par fil volant — Test non-validé après correction, mais cause localisée.**

8.5 Test 4 — Algorithme de comptage FSM

Procédure : Passage de la main devant le capteur de gauche à droite (entrée simulée), puis de droite à gauche (sortie simulée). Lecture de la variable passageCounter en mode Debug via STM32CubeIDE.

Résultat : Le compteur ne s'incrémente pas pour les raisons expliquées lors du test précédent. L'algorithme n'a donc pas pu être validé sur le PCB final. **Test non-validé.**

8.6 Récapitulatif

N°	Bloc testé	Méthode	Résultat
1	Alimentation 3,3 V	Multimètre	Validé
2	STM32 + LED D1	Flash + observation	Validé
3	Capteur I ² C 0x52	Console série + oscillo	Corrigé mais non-validé
4	Algorithme FSM	Debug CubeIDE	Non-validé
5	Sauvegarde EEPROM SPI	Lecture après reset	<i>En cours</i>

TABLE 8.1 – Récapitulatif des tests réalisés

9. Bilan, conseils et conclusion

9.1 Chronologie du projet

Période	Travaux réalisés
Fév. 2026 (S1–S2)	Rédaction du cahier des charges, TD KiCad, choix des composants
Mars 2026 (S3–S4)	Saisie du schéma électronique, vérification ERC
Mars–Avr. (S5–S6)	Routage PCB, vérification DRC, envoi des Gerbers à JLC PCB
Avr. 2026 (S7)	Réception du PCB, soudage des composants CMS
Mai 2026 (S8)	Tests bloc par bloc, identification et correction erreur I ² C
Mai 2026 (S9)	Détection des causes de nos problèmes, rédaction du rapport, préparation soutenance

TABLE 9.1 – Chronologie du projet — du cahier des charges à la soutenance

9.2 Trois conseils positifs pour nos successeurs

1. **Vérifier le pin mapping avant de router.** Croiser systématiquement chaque signal avec le tableau *Alternate Function Mapping* de la fiche technique STM32. Notre erreur PA15/PA8 sur l’I²C2 nous a coûté plusieurs heures de débogage.
2. **Imprimer le PCB à l’échelle 1 :1 avant de commander.** Poser physiquement les composants réels sur l’impression papier permet de vérifier les empreintes et de détecter les erreurs de taille avant fabrication.
3. **Commencer le code sur la carte Nucleo pendant que le PCB est en fabrication.** Les drivers ST de l’API VL53L5CX peuvent être testés indépendamment du PCB, ce qui évite de perdre du temps une fois les cartes reçues.

9.3 Trois points à éviter

1. **Ne pas travailler séquentiellement.** Attendre d’avoir terminé le schéma pour commencer le code, ou attendre la réception du PCB pour tester le logiciel, fait perdre un temps précieux. Matériel et logiciel doivent avancer en parallèle.
2. **Ne pas négliger les condensateurs de découplage.** Un condensateur mal placé ou oublié peut provoquer des resets aléatoires extrêmement difficiles à diagnostiquer.
3. **Ne pas accumuler les blocs non testés.** Tester au fur et à mesure (alimentation d’abord, puis MCU, puis chaque périphérique séparément) est infiniment plus efficace que de tout intégrer d’un coup.

9.4 Conclusion

Ce mini-projet nous a permis de parcourir **l’intégralité du cycle de développement d’un système embarqué** : cahier des charges, conception sous KiCad, fabrication industrielle (JLC PCB), soudage, programmation et validation.

Sur le plan technique, l’API VL53L5CX de STMicroelectronics s’est avérée bien documentée et relativement accessible, mais la gestion des statuts de mesure et le filtrage des données brutes

nécessitent une lecture attentive de la documentation. La machine à états finis s'est révélée être une architecture logicielle élégante, robuste et facilement extensible.

Sur le plan humain, ce projet nous a appris à **coordonner un travail en équipe de quatre**, à communiquer efficacement sur les blocages techniques et à prendre des décisions collectives sous contrainte de temps.

Les améliorations identifiées pour une prochaine révision sont : correction du routage I²C dans le schéma KiCad (Rev. 1.1), validation complète de la sauvegarde EEPROM après reset, et évolution de l'algorithme FSM pour gérer les passages simultanés de plusieurs personnes.

Références principales

- [1] STMicroelectronics, *VL53L5CX Product Overview*, DS13754, 2023.
- [2] STMicroelectronics, *VL53L5CX ULD API User Manual*, UM2884, 2023.
- [3] STMicroelectronics, *STM32G431KB Datasheet*, DS12589, 2022.
- [4] STMicroelectronics, *M95512-RMN6TP EEPROM Datasheet*, 2020.
- [5] ROHM Semiconductor, *BU33SD5WG-TR Datasheet*, 2021.
- [6] KiCad Documentation, consulté mai 2026.